

[L0-L1 Core Essence & Design Logic]

L0 Essence: : A relational computation system optimized for CPU and I/O throughput under the physical constraints of volatile memory and non-volatile storage, guaranteeing ACID invariants via concurrency control and write-ahead logging.

L1 Logic:
:
1. **Concurrent Thread/Process Scheduling** (M1) minimizes context-switching overheads, leveraging internal memory locks (latches/mutexes) to isolate concurrent memory access.
2. **Query Optimization & Execution** (M2) translates logical SQL queries into optimized physical plan trees using Cost-Based Optimizers (CBO) and executes them via Volcano iterators or vectorized loops.
3. **Buffer Pool Clock Replacement & Slotted Pages** (M3) align record locations on disk pages and manage cached memory buffers, smoothing out random disk I/O latencies.
4. **SS2PL Concurrency & ARIES Recovery** (M4 & M5) defend transaction boundaries, resolving wait-lock deadlocks and reconstructing crashed state spaces via three-phase WAL analysis.

[L2 Database Kernel Data Flow Topology]

:
• [SQL Client Request] → [Worker Dispatch] → [Parser AST & CBO Plan] → [Physical Executor Tree] → [SS2PL Lock Manager Check] → [Buffer Pool Page Pin] → [Physical Page Read/Write] → [WAL Log Flush & fsync] → [Dirty Page Clock Flush]

[M1.1] Process-per-Connection Model

- **Early Engine Architectures:** Oracle and PostgreSQL historically adopted multi-process designs, allocating a dedicated OS process for each client socket.
- **Inter-Process Overhead:** Inter-process communication (IPC) must route through shared memory blocks, and process context-switching consumes CPU, limiting active connections.

[M1.2] Thread-per-Connection Model

- **Shared Address Spaces:** MySQL and SQL Server allocate a dedicated thread per connection, sharing the parent process’s memory space.
- **Memory Footprint Limits:** Each thread reserves stack spaces, leading to memory fragmentation and high thread scheduler context-switches at scale.

[M1.3] Worker Pool Scheduling

- **Decoupled Connections:** Modern engines decouple connection sockets from execution threads by building fixed-size worker pools.
- **Multiplexed Runloop:** Frontend request routing utilizes async network syscalls (e.g., `epoll`) to queue execution tasks, processed by thread pools to scale connections efficiently.

[M1.4] Latch vs Mutex

- **Locking vs Syncing:** In databases, `Lock` protects logical transactions (user-visible concurrency), while `Latch` (mutexes, rwlocks) protects internal shared memory structures.
- **Transient Latencies:** Latches are lightweight SPIN locks implemented via CAS instructions, held for microseconds to sync page directories and buffer frames.

[M2.1] SQL Parsing & Rewriting

- **Abstract Syntax Tree:** The SQL parser parses text strings into an Abstract Syntax Tree (AST), checking keywords and schemas.
- **Semantic Simplification:** The rewriter flattens views, pre-evaluates constants, and simplifies logic to translate AST structures into logical execution plans.

[M2.2] Cost-Based Optimizer

- **Plan Cost Search:** The CBO evaluates alternative physical operators to select the path with the lowest overall hardware footprint.
- **Cost Evaluation Formula:** Uses statistics (histograms, row counts) to calculate plan weights: $Cost = Page_Reads + W_{IO} * Tuples_Scanned + W_{CPU}$, mapping out physical plans.

[M2.3] System R Join Plan Optimizer

- **Join Space Combinatorics:** Join orders scale exponentially with table counts. The System R optimizer uses dynamic programming to search plans bottom-up.
- **Left-Deep Tree Truncation:** Restricting queries to left-deep trees (where the right child must be a base table) crops the search space, optimizing multi-table join sequences.

[M2.4] Volcano Iterator Model

- **Standard Operator Contract:** The classic query engine pattern. Operators (e.g., `SeqScan`, `HashJoin`) expose a standard interface containing `open()`, `next()`, and `close()`.
- **Streaming Pipeline:** The parent operator calls `next()` to pull one tuple at a time from child operators, keeping memory overhead minimal.

[M2.5] Vectorized Execution vs JIT

- **Volcano Iterator Overhead:** Volcano query execution incurs frequent C++ virtual function calls per record, leading to CPU instruction cache misses.
- **Execution Evolutions:**
 1. **Vectorized Execution:** Operators process tuple arrays (e.g., 1024 records) in single runs, utilizing CPU L1 caches and SIMD registers.
 2. **JIT Compilation:** LLVM compiles execution trees into machine code, eliminating virtual calls.

[M3.1] Slotted-Page Storage Layout

- **In-Place Update Defense:** To prevent variable-length updates (e.g., `VARCHAR` edits) from shifting page offsets, databases use slotted page layouts.
- **Bidirectional Growth:** Page headers and slot pointer arrays grow downwards from the top, while tuple data grows upwards from the page bottom, optimizing row updates.

[M3.2] Clock Replacement Policy

- **LRU Mutex Bottlenecks:** LRU page replacement requires acquiring global mutexes to adjust head/tail pointers on page hits, causing lock contention.
- **Circular Eviction Hand:** The Clock algorithm maps buffer pool frames to a circular array. A hand sweeps frames, evicting frames with zero usage bits (`usage_count == 0`), bypassing lock contention.

[M3.3] Row-Store (NSM) vs Column-Store (DSM)

- **Storage Trade-offs:**
 1. **NSM (Row-Store):** Packs whole record attributes sequentially on pages, optimal for OLTP point updates and inserts.
 2. **DSM (Column-Store):** Groups column values together, optimizing OLAP aggregation queries by only loading required attributes.

[M3.4] Buffer Prefetching & Double Buffering

- **Masking I/O Latency:** Waiting on disk I/O stalls query operators.
- **Asynchronous Prefetches:** The storage manager predicts future page lookups and pre-loads pages into the buffer pool asynchronously, using double buffering to overlap compute with disk wait times.

🔒 Database Concurrency & Recovery Trade-offs & Fallacies Matrix

Fine-grained row-level locks enable maximum concurrency with no downsides.: Thousands of row locks exhaust lock manager memory. Frequent lock escalations (**Lock Escalation**) force row locks to merge into table locks, triggering transaction-level deadlocks. → Limit total lock allocations. When row locks exceed thresholds, escalate them. Use optimistic concurrency control (OCC) or versioned MVCC to bypass write conflicts in high-contention systems.

Crash recovery can simply roll back all uncommitted transactions.: Directly rolling back updates corrupts dirty page indexes on disk, leaving database files half-synchronized. The ARIES protocol requires “repeating history” (redo) first. → Enforce ARIES three-phase recovery: first scan logs (Analysis), redo all logged modifications (including uncommitted ones) up to the crash point, and then rollback (Undo) uncommitted actions.

LRU lists are the perfect choice for page replacement policies.: Under high concurrency, constantly moving nodes to the head/tail of an LRU double-linked list requires acquiring global mutexes (latches), bottlenecking the buffer pool eviction thread. → Enable Clock replacement. Organize buffer pages in a circular array, using usage counters (`usage_count`) and a sweeping hand to evict pages without heavy lock contention.

[M4.1] Strict Two-Phase Locking (SS2PL)

- **Serializability Invariant:** SS2PL requires transactions to acquire locks during a growing phase and release all exclusive locks (X Locks) only upon commit or rollback.
- **Cascading Rollback Prevention:** Holding write locks until commit prevents other transactions from reading uncommitted modifications, eliminating cascading rollbacks.

[M4.2] Lock Manager Layout

- **In-Memory Lock Directory:** The lock manager maintains a global hash table mapping resources to lock structures.
- **Double-Linked Lists:** Lock headers contain resource IDs and link two lists: `Granted List` (holding transactions) and `Wait List` (queued transactions), validating lock states.

[M4.3] Lock Escalation & Deadlock Detection

- **Escalation Safety:** When fine-grained locks (row locks) overwhelm lock tables, the lock manager merges them into coarser locks (table locks) to save memory.
- **Deadlock Resolution:** The manager builds a Wait-For-Graph and checks for cycles. If a cycle is detected, it aborts a victim transaction, releasing its locks.

[M4.4] MVCC Version Chain

- **Non-Blocking Reads:** Write transactions write mutations as version deltas to Undo Logs rather than overwriting database records.
- **Read View Visibility:** Readers query using a `Read View` snapshot, traversing version chains to locate the latest committed record, isolating reads from write stalls.

[M4.5] OCC Validation

- **Lockless Execution:** OCC assumes write conflicts are rare, executing writes to private workspaces.
- **Validation Check:** During validation, the engine verifies that read records were not updated by concurrent transactions, committing changes only on validation success.

[M5.1] WAL Protocol

- **Durability Contract:** Modified database pages must not be written to disk until the corresponding log entry is flushed.
- **Transaction Durability:** Commit operations block until all associated WAL records are flushed via `fsync`, ensuring crash recovery.

[M5.2] ARIES Analysis Phase

- **Recovery Startup:** Analysis starts at the last checkpoint.
- **State Reconstruction:** The engine scans logs forward to rebuild the active Transaction Table (TT) and Dirty Page Table (DPT), identifying the oldest unwritten page LSN.

[M5.3] ARIES Redo Phase

- **Repeating History:** Redo starts at the oldest unwritten page LSN, positive-scanning logs to replay all logged modifications (committed or uncommitted), aligning the disk database state.

[M5.4] ARIES Undo Phase

- **Log Cleanup:** Undo rolls back transactions left uncommitted at the crash point.
- **Reverse Scan rollback:** Scans logs backward to reverse modifications, writing Compensation Log Records (CLRs) to prevent redundant rollbacks if re-crashes occur.

[M5.5] Fuzzy Checkpointing

- **Non-Blocking Checkpoints:** Writing checkpoints must not block write transactions.
- **Incremental Checkpoints:** Fuzzy checkpoints log active DPT and TT states to log files without forcing page flushes, enabling check operations to run concurrently with writes.

[M6.1] Shared-Memory Architecture

- **Bus Performance Limits:** Shared-memory systems connect CPU nodes to a single physical RAM and disk stack.
- **Interconnect Bottlenecks:** As CPU count grows, memory bus collisions and cache consistency overheads scale, capping hardware scaling.

[M6.2] Shared-Disk Architecture

- **SAN Disk Interconnect:** Nodes carry dedicated memory but share a central SAN storage array.
- **DLM Overhead:** To prevent data corruption on concurrent writes, nodes coordinate pages via a Distributed Lock Manager (DLM), introducing latency.

[M6.3] Shared-Nothing Architecture

- **Independent Nodes:** Each node runs dedicated CPUs, memory, and disks, communicating only over high-speed networks.
- **Scale-Out Capacity:** Data partitions (shards) distribute queries across nodes, allowing modern distributed databases (e.g., CockroachDB, ClickHouse) to scale out.

[M6.4] Two-Phase Commit (2PC)

- **Distributed Commit Invariant:**
 1. **Prepare Phase:** The coordinator asks participants to prepare. Nodes write updates to WAL, lock resources, and return Ready.
 2. **Commit Phase:** If all nodes return Ready, the coordinator broadcasts commit; otherwise, it aborts, ensuring atomic consistency.

[M6.5] Distributed Join & Data Redistribution

- **Join Network Footprints:** Multi-node joins require transferring records over networks.
- **Join Optimizations:**
 1. **Colocated Join:** Joins tables pre-partitioned on join keys locally.
 2. **Broadcast Join:** Copies smaller tables to all node segments.
 3. **Shuffle Join:** Hashes and redistributes tables across nodes.

🔧 Zone T: Database Performance & Diagnostic Labs

T1 Database System Tuning Thresholds

```
:-::: (---)
:-
shared_buffers (25% ~ 40% of RAM)
Allocates database page cache buffers. Setting this too high risks exhausting host OS memory and triggers double-caching stalls.
max_connections (100 ~ 500 (use pools above))
Caps concurrent connection sockets. Excess connections cause high context-switching overheads.
lock_timeout (1000 ms ~ 5000 ms)
Timeout threshold for acquiring transaction locks, preventing deadlock hangs.
wal_buffers (32 MB ~ 64 MB)
In-memory buffer size for unwritten WAL records, optimizing batch writes.
```

T2 Database Maintenance Commands

```
• 1. Active Query & Lock Diagnostics
sql
-- PostgreSQL: View active non-idle queries and runtimes
SELECT pid, usename, query, state, age(clock_timestamp(), query_start) AS duration
FROM pg_stat_activity
WHERE state != 'idle' ORDER BY duration DESC;

• 2. Lock Contention Analysis
sql
-- PostgreSQL: Check blocked and blocking transactions
SELECT a.pid AS blocked_pid, a.query AS blocked_query,
       l.locktype, l.mode, l.granted,
       d.pid AS blocking_pid, d.query AS blocking_query
FROM pg_catalog.pg_locks l
JOIN pg_catalog.pg_stat_activity a ON l.pid = a.pid
JOIN pg_catalog.pg_locks blocked_by ON l.locktype = blocked_by.locktype
AND l.database IS NOT DISTINCT FROM blocked_by.database
AND l.relation IS NOT DISTINCT FROM blocked_by.relation
AND l.page IS NOT DISTINCT FROM blocked_by.page
AND l.tuple IS NOT DISTINCT FROM blocked_by.tuple
AND l.virtualxid IS NOT DISTINCT FROM blocked_by.virtualxid
AND l.transactionid IS NOT DISTINCT FROM blocked_by.transactionid
AND l.classid IS NOT DISTINCT FROM blocked_by.classid
AND l.objid IS NOT DISTINCT FROM blocked_by.objid
AND l.objsubid IS NOT DISTINCT FROM blocked_by.objsubid
AND l.pid != blocked_by.pid
JOIN pg_catalog.pg_stat_activity d ON blocked_by.pid = d.pid
WHERE NOT l.granted;

• 3. InnoDB Lock Monitor
sql
-- MySQL: Show transaction history, locks, and latch states
SHOW ENGINE INNODB STATUS;
```